

最大社交距离

题目描述

疫情期间需要大家保证一定的 **社交距离**，公司组织开交流会议。

座位一排共 N 个座位，编号分别为 $[0, N - 1]$ 。

要求员工一个接着一个进入会议室，并且可以在任何时候离开会议室。

满足：

- 每当一个员工进入时，需要坐到最大社交距离（最大化自己和其他人的距离的座位）；
- 如果有多个这样的座位，则坐到索引最小的那个座位。

输入描述

会议室座位 总数 seatNum

- $1 \leq \text{seatNum} \leq 500$

员工的进出顺序 seatOrLeave 数组

- 元素值为 1，表示进场
- 元素值为负数，表示出场（特殊：位置 0 的员工不会离开）

例如 -4 表示坐在位置 4 的员工离开（保证有员工坐在该座位上）

输出描述

最后进来员工，他会坐在第几个位置，如果位置已满，则输出-1。

用例

输入	10 [1, 1, 1, 1, -4, 1]
输出	5
说明	seat -> 0,空在任何位置都行，但是要给他安排索引最小的位置，也就是座位 0 seat -> 9,要和旁边的人距离最远，也就是座位 9 seat -> 4,要和旁边的人距离最远，应该坐到中间，也就是座位 4 seat -> 2,员工最后坐在 2 号座位上 leave[4], 4 号座位的员工离开 seat -> 5,员工最后坐在 5 号座位上

题目解析

我的解题思路如下：

首先，定义一个集合 seatIdx 用于记录已经坐人的座位号。

然后，遍历第二行输入的员工出入顺序数组，如果遍历出来的数组元素info：

- info值为负数，则坐在 -info 座位号的员工要离开，我们应该将 -info 从 seatIdx 中去除。
- info值为正数，则有一个员工要进来入座，我们需要找到具有最大社交距离的座位给这个员工：

假设座位使用情况如：[1, 0, 0, 1, 0 , 0, 0]，其中1代表有人坐，0代表没人坐。

我们观察其中的连续空闲座位情况，连续空闲座位的左右边界有如下情况：

- 1、左右边界都无人，此时必然是所有座位都为空：[0, 0, 0, 0, 0]，因为题目说：位置 0 的员工不会离开
- 2、左右边界都有人：[1, 0, 0, 1, 0 , 0, 0]

3、左边界有人，右边界无人：[1, 0, 0, 1, 0, 0, 0]

4、左边界无人，右边界有人，本题不存在这种情况，因为题目说：位置 0 的员工不会离开

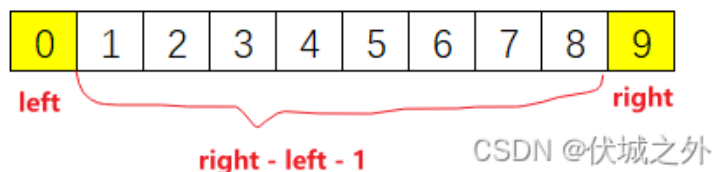
如果有员工要入座，则检查：

如果seatIdx.size == 0，则说明所有座位都是空的，此时对应员工直接做到第0个座位

如果seatIdx.size == 1，则说明只有一个座位坐了人，那么该座位肯定是第0个座位，因此当前要入座的员工，最大社交距离位置是第 seatNum - 1 个位置

如果seatIdx.size > 1，则此时我们需要遍历seatIdx，即遍历出哪些座位做了人？一次遍历两个，即获得了连续空闲座位的左右边界，假设左边界是left，右边界是right，

那么连续空闲座位长度 $dis = right - left - 1$ ，如下图所示：



此时该连续空闲座位中选一个具有最大社交距离的位置，通过图示，我们可以很容易判断出是第4个位置。

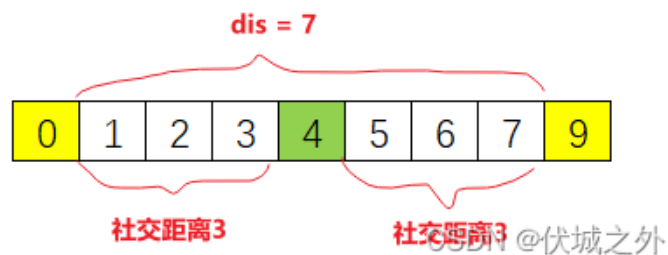
求解时，我们可以先根据dist求出最大社交距离curSeatDis为：

$curSeatDis = dis / 2$

如果 dis 是偶数，则还需要： $curSeatDis -= 1$ ，比如下图 $dis = 8$ ，座位4的社交距离 $= 8 / 2 - 1 = 3$



如果 dis 是奇数，则不需要做处理，比如下图：



得到当前空闲区域的最大社交距离后，我们即可求出当前空闲区域具备最大社交距离的座位号：

$\text{curSeatIdx} = \text{left} + \text{curSeatDis} + 1$

如果当前座位情况存在多个连续空闲座位区域，比如 **[1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1]**

此时，我们应该按照上面逻辑，求出每一个空闲区域的最大社交距离，如果对应空闲区域的最大社交距离更大，则对应空闲区域可以得到更优的座位。

题目说：

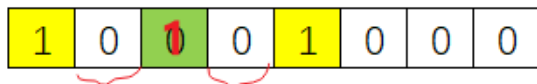
如果有多个这样的座位，则坐到索引最小的那个座位。

因此，只有当前空闲区域的最大社交距离严格大于前面的，才能更新最优座位号。这样可以保证出现相同最大社交距离时，可以保留索引较小的座位号。

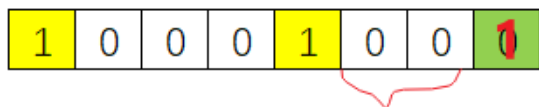
另外，还有一种特殊情况，比如下面座位使用情况

[1, 0, 0, 0, 1, 0, 0, 0]

此时进来一个员工入座的话，则选择最后一个座位，社交距离更大



如果选这里，则社交距离为1 CSDN @伏城之外



如果选这里，则社交距离为2
CSDN @伏城之外

即，如果最后一个座位空闲，那么选择坐最后一个座位的社交距离计算公式是不同于之前左右边界都有人的情况的，此时最大社交距离curSeatDis为：

```
curSeatDis = seatNum - 1 - seatIdx.getLast - 1
```

我们应该考虑到这种情况。

另外，本题还需要考虑坐满的情况。更多细节请看代码实现。

JS算法源码

```
1 const rl = require("readline").createInterface({ input: process.stdin });
2 var iter = rl[Symbol.asyncIterator]();
3 const readline = async () => (await iter.next()).value;
4
5 void (async function () {
6   const seatNum = parseInt(await readline());
7   const seatOrLeave = JSON.parse(await readline());
```

运行

```
8 console.log(getResult(seatNum, seatOrLeave)); 9 |})();
10
11 function getResult(seatNum, seatOrLeave) {
12     // 记录已经坐人位置的序号
13     let seatIdx = [];
14
15     // 记录题解
16     let lastSeatIdx = -1;
17
18     // 遍历员工的进出顺序
19     for (let info of seatOrLeave) {
20         // 如果当前元素值为负数，表示出场（特殊：位置 0 的员工不会离开）
21         // 例如 -4 表示坐在位置 4 的员工离开（保证有员工坐在该座位上）
22         if (info < 0) {
23             const leaveIdx = -info;
24             seatIdx = seatIdx.filter((idx) => idx !== leaveIdx);
25             continue;
26         }
27
28         // 如果当前元素值为 1，表示进场
29         // 如果没有空闲位置，则坐不下
30         if (seatIdx.length === seatNum) {
31             lastSeatIdx = -1;
32             continue;
33         }
34
35         if (seatIdx.length === 0) {
36             // 当前人员进场前，座位上没有人，则当前人员是第一个进场的，直接坐第0个位置
37             seatIdx.push(0);
38             lastSeatIdx = 0;
39         } else if (seatIdx.length === 1) {
40             // 当前人员进场前，座位上只有一个人，那么这个人肯定坐在第0个位置，则当前进场的人坐在 seatNum - 1 位置才能离 0 位置最远
41             seatIdx.push(seatNum - 1);
42             lastSeatIdx = seatNum - 1;
43         } else {
44             // 记录具有最大社交距离的座位号
45             let bestSeatIdx = -1;
```

```

46 // 记录最大社交距离
47     let bestSeatDis = -1;
48
49 let left = seatIdx[0]; // 左边界
50 for (let i = 1; i < seatIdx.length; i++) {
51     const right = seatIdx[i]; // 右边界
52
53     // 连续空闲座位区域的长度
54     const dis = right - left - 1;
55
56     // 如果连续空闲座位区域长度为0，则无法坐人，此时遍历下一个连续空闲座位区域
57     // 如果连续空闲座位区域长度大于0，则可以坐人
58     if (dis > 0) {
59         // 当前空闲区域能产生的最大社交距离
60         const curSeatDis = Math.floor(dis / 2) - (dis % 2 == 0 ? 1 : 0);
61         // 当前空闲区域中具备最大社交距离的位置
62         const curSeatIdx = left + curSeatDis + 1;
63
64         // 保留最优解
65         if (curSeatDis > bestSeatDis) {
66             bestSeatDis = curSeatDis;
67             bestSeatIdx = curSeatIdx;
68         }
69     }
70
71     left = right;
72 }
73
74 // 如果最后一个座位，即第 seatNum - 1 号座位没有坐人的话，比如 1 0 0 0 1 0 0 0 0，此时最后一段空闲区域是没有右边界的，需要特殊处理
75 if (seatIdx.at(-1) < seatNum - 1) {
76     // 此时可以直接坐到第 seatNum - 1 号座位，最大社交距离为 curSeatDis
77     const curSeatDis = seatNum - 1 - seatIdx.at(-1) - 1;
78     const curSeatIdx = seatNum - 1;
79
80     // 保留最优解
81     if (curSeatDis > bestSeatDis) {
82         bestSeatIdx = curSeatIdx;

```

```

83         }84     }
85
86     // 如果能坐人，则将坐的位置加入seatIdx中
87     if (bestSeatIdx > 0) {
88         seatIdx.push(bestSeatIdx);
89         seatIdx.sort((a, b) => a - b);
90     }
91
92     // 假设当前人就是最后一个人，那么无论当前人是否能坐进去，都更新lastSeatIdx = bestSeatIdx
93     lastSeatIdx = bestSeatIdx;
94 }
95 }
96
97 return lastSeatIdx;
98 }

```

Java算法源码

```

1  import java.util.ArrayList;
2  import java.util.Arrays;
3  import java.util.Scanner;
4
5  public class Main {
6      public static void main(String[] args) {
7          Scanner sc = new Scanner(System.in);
8
9          int seatNum = Integer.parseInt(sc.nextLine());
10
11         String tmp = sc.nextLine();
12         int[] searOrLeave =
13             Arrays.stream(tmp.substring(1, tmp.length() - 1).split(", "))
14                 .mapToInt(Integer::parseInt)
15                 .toArray();

```



```
16 | 17 | System.out.println(getResult(seatNum, seatOrLeave));
18 | }
19 |
20 | public static int getResult(int seatNum, int[] seatOrLeave) {
21 |     // 记录已经坐人位置的序号
22 |     ArrayList<Integer> seatIdx = new ArrayList<>();
23 |
24 |     // 记录题解
25 |     int lastSeatIdx = -1;
26 |
27 |     // 遍历员工的进出顺序
28 |     for (int info : seatOrLeave) {
29 |         // 如果当前元素值为负数，表示出场（特殊：位置 0 的员工不会离开）
30 |         // 例如 -4 表示坐在位置 4 的员工离开（保证有员工坐在该座位上）
31 |         if (info < 0) {
32 |             int leaveIdx = -info;
33 |             seatIdx.remove(new Integer(leaveIdx));
34 |             continue;
35 |         }
36 |
37 |         // 如果当前元素值为 1，表示进场
38 |         // 如果没有空闲位置，则坐不下
39 |         if (seatIdx.size() == seatNum) {
40 |             // 假设当前人就是最后一个人
41 |             lastSeatIdx = -1;
42 |             continue;
43 |         }
44 |
45 |         if (seatIdx.size() == 0) {
46 |             // 当前人员进场前，座位上没有人，则当前人员是第一个进场的，直接坐第0个位置
47 |             seatIdx.add(0);
48 |             lastSeatIdx = 0;
49 |         } else if (seatIdx.size() == 1) {
50 |             // 当前人员进场前，座位上只有一个人，那么这个人肯定坐在第0个位置，则当前进场的人坐在 seatNum - 1 位置才能离 0 位置最远
51 |             seatIdx.add(seatNum - 1);
52 |             lastSeatIdx = seatNum - 1;
53 |         } else {
```

```

54 // 记录具有最大社交距离的座位号
55 int bestSeatIdx = -1;

56 // 记录最大社交距离
57 int bestSeatDis = -1;
58
59 // 找到连续空闲座位区域（该区域左、右边界是坐了人的座位）
60
61 int left = seatIdx.get(0); // 左边界
62 for (int i = 1; i < seatIdx.size(); i++) {
63     int right = seatIdx.get(i); // 右边界
64
65     // 连续空闲座位区域的长度
66     int dis = right - left - 1;
67
68     // 如果连续空闲座位区域长度为0，则无法坐人，此时遍历下一个连续空闲座位区域
69     // 如果连续空闲座位区域长度大于0，则可以坐人
70     if (dis > 0) {
71         // 当前空闲区域能产生的最大社交距离
72         int curSeatDis = dis / 2 - (dis % 2 == 0 ? 1 : 0);
73         // 当前空闲区域中具备最大社交距离的位置
74         int curSeatIdx = left + curSeatDis + 1;
75
76         // 保留最优解
77         if (curSeatDis > bestSeatDis) {
78             bestSeatDis = curSeatDis;
79             bestSeatIdx = curSeatIdx;
80         }
81     }
82
83     left = right;
84 }
85
86 // 如果最后一个座位，即第 seatNum - 1 号座位没有坐人的话，比如 1 0 0 0 1 0 0 0 0，此时最后一段空闲区域是没有右边界的，需要特殊处理
87 if (seatIdx.get(seatIdx.size() - 1) < seatNum - 1) {
88     // 此时可以直接坐到第 seatNum - 1 号座位，最大社交距离为 curSeatDis
89     int curSeatDis = seatNum - 1 - seatIdx.get(seatIdx.size() - 1) - 1;
90     int curSeatIdx = seatNum - 1;

```

```

91 |
92 |         // 保留最优解
93 |         if (curSeatDis > bestSeatDis) {
94 |             bestSeatIdx = curSeatIdx;
95 |         }
96 |     }
97 |
98 |     // 如果能坐人，则将坐的位置加入seatIdx中
99 |     if (bestSeatIdx > 0) {
100 |         seatIdx.add(bestSeatIdx);
101 |         seatIdx.sort((a, b) -> a - b);
102 |     }
103 |
104 |     // 假设当前人就是最后一个人，那么无论当前人是否能坐进去，都更新lastSeatIdx = bestSeatIdx
105 |     lastSeatIdx = bestSeatIdx;
106 | }
107 | }
108 |
109 | return lastSeatIdx;
110 | }
111 | }

```

Python算法源码

```

1 | # 输入获取
2 | seatNum = int(input())
3 | seatOrLeave = eval(input())
4 |
5 |
6 | # 算法入口
7 | def getResult():
8 |     # 记录已经坐人位置的序号
9 |     seatIdx = []
10 |
11 |     # 记录题解

```

```
12 lastSeatIdx = -1
13
14 # 遍历员工的进出顺序
15 for info in seatOrLeave:
16     # 如果当前元素值为负数，表示出场（特殊：位置 0 的员工不会离开）
17     # 例如 -4 表示坐在位置 4 的员工离开（保证有员工坐在该座位上）
18     if info < 0:
19         leaveIdx = -info
20         seatIdx.remove(leaveIdx)
21         continue
22
23     # 如果当前元素值为 1，表示进场
24     # 如果没有空闲位置，则坐不下
25     if len(seatIdx) == seatNum:
26         lastSeatIdx = -1
27         continue
28
29     if len(seatIdx) == 0:
30         # 当前人员进场前，座位上没有人，则当前人员是第一个进场的，直接坐第0个位置
31         seatIdx.append(0)
32         lastSeatIdx = 0
33     elif len(seatIdx) == 1:
34         # 当前人员进场前，座位上只有一个人，那么这个人肯定坐在第0个位置，则当前进场的人坐在 seatNum - 1 位置才能离 0 位置最远
35         seatIdx.append(seatNum - 1)
36         lastSeatIdx = seatNum - 1
37     else:
38         # 记录具有最大社交距离的座位号
39         bestSeatIdx = -1
40         # 记录最大社交距离
41         bestSeatDis = -1
42
43         # 找到连续空闲座位区域（该区域左、右边界是坐了人的座位）
44
45         left = seatIdx[0] # 左边界
46         for i in range(1, len(seatIdx)):
47             right = seatIdx[i] # 右边界
48
49             # 连续空闲座位区域的长度
```

```

50         dis = right - left - 1
51     # 如果连续空闲座位区域长度为0，则无法坐人，此时遍历下一个连续空闲座位区域
52     # 如果连续空闲座位区域长度大于0，则可以坐人
53     if dis > 0:
54         # 当前空闲区域能产生的最大社交距离
55         curSeatDis = dis // 2 - (1 if dis % 2 == 0 else 0)
56         # 当前空闲区域中具备最大社交距离的位置
57         curSeatIdx = left + curSeatDis + 1
58
59         # 保留最优解
60         if curSeatDis > bestSeatDis:
61             bestSeatDis = curSeatDis
62             bestSeatIdx = curSeatIdx
63
64     left = right
65
66     # 如果最后一个座位，即第 seatNum - 1 号座位没有坐人的话，比如 1 0 0 0 1 0 0 0 0，此时最后一段空闲区域是没有右边界的，需要特殊处理
67     if seatIdx[-1] < seatNum - 1:
68         # 此时可以直接坐到第 seatNum - 1 号座位，最大社交距离为 curSeatDis
69         curSeatDis = seatNum - 1 - seatIdx[-1] - 1
70         curSeatIdx = seatNum - 1
71
72         # 保留最优解
73         if curSeatDis > bestSeatDis:
74             bestSeatIdx = curSeatIdx
75
76     # 如果能坐人，则将坐的位置加入seatIdx中
77     if bestSeatIdx > 0:
78         seatIdx.append(bestSeatIdx)
79         seatIdx.sort()
80
81     # 假设当前人就是最后一个人，那么无论当前人是否能坐进去，都更新lastSeatIdx = bestSeatIdx
82     lastSeatIdx = bestSeatIdx
83
84     return lastSeatIdx
85
86

```

```
87 |  
88 | # 算法调用  
89 | print(getResult())
```

C算法源码

```
1  #include <stdio.h>  
2  #include <stdlib.h>  
3  
4  #define MAX_SIZE 500  
5  
6  int cmp(const void *a, const void *b) {  
7      return *((int *) a) - *((int *) b);  
8  }  
9  
10 int main() {  
11     int seatNum;  
12     scanf("%d", &seatNum);  
13  
14     getchar();  
15  
16     int seatOrLeave[MAX_SIZE] = {0};  
17     int seatOrLeave_size = 0;  
18  
19     while (scanf("[%d", &seatOrLeave[seatOrLeave_size]) || scanf("%d", &seatOrLeave[seatOrLeave_size])) {  
20         seatOrLeave_size++;  
21         if (getchar() != ',') break;  
22     }  
23  
24     // 记录已经坐人位置的序号  
25     int seatIdx[MAX_SIZE];  
26     int seatIdx_size = 0;  
27  
28     // 记录题解  
29     int lastSeatIdx = -1;
```

运行

```

30
31 | // 遍历员工的进出顺序
32 for (int i = 0; i < seatOrLeave_size; i++) {
33     int info = seatOrLeave[i];
34
35     // 如果当前元素值为负数，表示出场（特殊：位置 0 的员工不会离开）
36     // 例如 -4 表示坐在位置 4 的员工离开（保证有员工坐在该座位上）
37     if (info < 0) {
38         int leaveIdx = -info;
39
40         // 删除seatIdx中对应leaveIdx
41         int j = 0;
42         for (; j < seatIdx_size; j++) {
43             if (seatIdx[j] == leaveIdx) break;
44         }
45
46         for (; j < seatIdx_size - 1; j++) {
47             seatIdx[j] = seatIdx[j + 1];
48         }
49
50         seatIdx_size--;
51
52         continue;
53     }
54
55     // 如果当前元素值为 1，表示进场
56     // 如果没有空闲位置，则坐不下
57     if (seatIdx_size == seatNum) {
58         // 假设当前人就是最后一个人
59         lastSeatIdx = -1;
60         continue;
61     }
62
63     if (seatIdx_size == 0) {
64         // 当前人员进场前，座位上没有人，则当前人员是第一个进场的，直接坐第0个位置
65         seatIdx[seatIdx_size++] = 0;
66         lastSeatIdx = 0;

```

```

67 | } else if (seatIdx_size == 1) { 68 |
    | // 当前人员进场前，座位上只有一个人，那么这个人肯定坐在第0个位置，则当前进场的人坐在 seatNum - 1 位置才能离 0 位置最远 69 |
    | seatIdx[seatIdx_size++] = seatNum - 1; 70 | lastSeatIdx = seatNum - 1;
71 | } else {
72 |     // 记录具有最大社交距离的座位号
73 |     int bestSeatIdx = -1;
74 |     // 记录最大社交距离
75 |     int bestSeatDis = -1;
76 |
77 |     // 找到连续空闲座位区域（该区域左、右边界是坐了人的座位）
78 |
79 |     int left = seatIdx[0]; // 左边界
80 |     for (int j = 1; j < seatIdx_size; j++) {
81 |         int right = seatIdx[j]; // 右边界
82 |
83 |         // 连续空闲座位区域的长度
84 |         int dis = right - left - 1;
85 |
86 |         // 如果连续空闲座位区域长度为0，则无法坐人，此时遍历下一个连续空闲座位区域
87 |         // 如果连续空闲座位区域长度大于0，则可以坐人
88 |         if (dis > 0) {
89 |             // 当前空闲区域能产生的最大社交距离
90 |             int curSeatDis = dis / 2 - (dis % 2 == 0 ? 1 : 0);
91 |             // 当前空闲区域中具备最大社交距离的位置
92 |             int curSeatIdx = left + curSeatDis + 1;
93 |
94 |             // 保留最优解
95 |             if (curSeatDis > bestSeatDis) {
96 |                 bestSeatDis = curSeatDis;
97 |                 bestSeatIdx = curSeatIdx;
98 |             }
99 |         }
100 |
101 |         left = right;
102 |     }
103 |
104 |     // 如果最后一个座位，即第 seatNum - 1 号座位没有坐人的话，比如 1 0 0 0 1 0 0 0 0，此时最后一段空闲区域是没有右

```



```

105     if (seatIdx[seatIdx_size - 1] < seatNum - 1) {
106         // 此时可以直接坐到第 seatNum - 1 号座位，最大社交距离为 curSeatDis
107         int curSeatDis = seatNum - 1 - seatIdx[seatIdx_size - 1] - 1;
108         int curSeatIdx = seatNum - 1;
109
110         // 保留最优解
111         if (curSeatDis > bestSeatDis) {
112             bestSeatIdx = curSeatIdx;
113         }
114     }
115
116     // 如果能坐人，则将坐的位置加入 seatIdx 中
117     if (bestSeatIdx > 0) {
118         seatIdx[seatIdx_size++] = bestSeatIdx;
119         qsort(seatIdx, seatIdx_size, sizeof(int), cmp);
120     }
121
122     // 假设当前人就是最后一个人，那么无论当前人是否能坐进去，都更新 lastSeatIdx = bestSeatIdx
123     lastSeatIdx = bestSeatIdx;
124 }
125 }
126
127 printf("%d\n", lastSeatIdx);
128
129 return 0;
130 }

```

C++ 算法源码

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int solution(int seatNum, vector<int> &seatOrLeave) {
5      // 记录已经坐人位置的序号

```

```

6      vector<int> seatIdx;
7
8      // 记录题解
9      int lastSeatIdx = -1;
10
11     // 遍历员工的进出顺序
12     for (const auto &info: seatOrLeave) {
13         // 如果当前元素值为负数，表示出场（特殊：位置 0 的员工不会离开）
14         // 例如 -4 表示坐在位置 4 的员工离开（保证有员工坐在该座位上）
15         if (info < 0) {
16             int leaveIdx = -info;
17             seatIdx.erase(remove(seatIdx.begin(), seatIdx.end(), leaveIdx), seatIdx.end());
18             continue;
19         }
20
21         // 如果当前元素值为 1，表示进场
22         // 如果没有空闲位置，则坐不下
23         if (seatIdx.size() == seatNum) {
24             // 假设当前人就是最后一个人
25             lastSeatIdx = -1;
26             continue;
27         }
28
29         if (seatIdx.empty()) {
30             // 当前人员进场前，座位上没有人，则当前人员是第一个进场的，直接坐第0个位置
31             seatIdx.emplace_back(0);
32             lastSeatIdx = 0;
33         } else if (seatIdx.size() == 1) {
34             // 当前人员进场前，座位上只有一个人，那么这个人肯定坐在第0个位置，则当前进场的人坐在 seatNum - 1 位置才能离 0 位置最远
35             seatIdx.emplace_back(seatNum - 1);
36             lastSeatIdx = seatNum - 1;
37         } else {
38             // 记录具有最大社交距离的座位号
39             int bestSeatIdx = -1;
40             // 记录最大社交距离
41             int bestSeatDis = -1;
42

```

```
43 // 找到连续空闲座位区域（该区域左、右边界是坐了人的座位） 44
45 // 左边界
46 int left = seatIdx[0];
47 for (int i = 1; i < seatIdx.size(); i++) {
48     // 右边界
49     int right = seatIdx[i];
50
51     // 连续空闲座位区域的长度
52     int dis = right - left - 1;
53
54     // 如果连续空闲座位区域长度为0，则无法坐人，此时遍历下一个连续空闲座位区域
55     // 如果连续空闲座位区域长度大于0，则可以坐人
56     if (dis > 0) {
57         // 当前空闲区域能产生的最大社交距离
58         int curSeatDis = dis / 2 - (dis % 2 == 0 ? 1 : 0);
59         // 当前空闲区域中具备最大社交距离的位置
60         int curSeatIdx = left + curSeatDis + 1;
61
62         // 保留最优解
63         if (curSeatDis > bestSeatDis) {
64             bestSeatDis = curSeatDis;
65             bestSeatIdx = curSeatIdx;
66         }
67     }
68
69     left = right;
70 }
71
72 // 如果最后一个座位，即第 seatNum - 1 号座位没有坐人的话，比如 1 0 0 0 1 0 0 0 0，此时最后一段空闲区域是没有右边界的，需要特殊处理
73 if (seatIdx.back() < seatNum - 1) {
74     // 此时可以直接坐到第 seatNum - 1 号座位，最大社交距离为 curSeatDis
75     int curSeatDis = seatNum - 1 - seatIdx.back() - 1;
76     int curSeatIdx = seatNum - 1;
77
78     // 保留最优解
79     if (curSeatDis > bestSeatDis) {
80         bestSeatIdx = curSeatIdx;
```

```

81         }
82     }
83
84     // 如果能坐人，则将坐的位置加入seatIdx中
85     if (bestSeatIdx > 0) {
86         seatIdx.emplace_back(bestSeatIdx);
87         sort(seatIdx.begin(), seatIdx.end());
88     }
89
90     // 假设当前人就是最后一个人，那么无论当前人是否能坐进去，都更新lastSeatIdx = bestSeatIdx
91     lastSeatIdx = bestSeatIdx;
92 }
93 }
94
95 return lastSeatIdx;
96 }
97
98 int main() {
99     int seatNum;
100     cin >> seatNum;
101
102     cin.ignore();
103
104     string s;
105     getline(cin, s);
106
107     for (auto &c: s) {
108         if (c == ',' || c == '[' || c == ']') c = ' ';
109     }
110
111     vector<int> seatOrLeave;
112
113     stringstream ss(s);
114     string token;
115
116     while (getline(ss, token, ' ')) {
117         if (token.empty()) continue;

```

```
118         seatOrLeave.emplace_back(stoi(token));119     }
120
121     cout << solution(seatNum, seatOrLeave) << endl;
122
123     return 0;
124 }
```